

CIS270A Project 6 Requirements

Interactive File Maintenance

Wibaux University wants you to write an interactive file maintenance program for its instructor file.

WUINSTP Record Layout:

Record Format INSTREC

Field	Description	Length/Type	(Decimal positions)	Notes
INSTNO	Instructor Social Security number	9S	--	UNIQUE
IFNAME	First name	10	--	--
ILNAME	Last name	15	--	--
DEPT	Department	3	--	--
SALARY	Salary	8S	(2)	--
RANK	Academic rank	1	--	1 = instructor; 2 = assistant professor; 3 = associate professor; 4 = full professor
SEX	Gender	1	--	M = male; F = female
HIRDAT	Date of hire	8S	(0)	ccymmdd
MARSTS	Marital status	1	--	M = married; S = single
DEPEND	Number of dependents	2S	(0)	
TENURE	Tenured faculty	1	--	Y = yes; N = no
TITLE	Preferred title	1	--	1 = Dr.; 2 = Mr.; 3 = Mrs.; 4 = Ms.
STREET	Street address	20	--	--
CITY	City	15	--	--
STATE	State	2	--	--
ZIP	Postal code	9S	(0)	--
IEMAIL	Instructor email	30	--	--

Go to next page

First Screen:

10	20	30	40	50	60	70	80
		Wibaux University				MM/DD/YY	
		Instructor File Maintenance					
		Your name here					
		Social Security number . . 999999999					
		Action code I					
		A = add record					
		C = change record					
		D = delete record					
		Type values, then Enter					
20		F3=Exit					

Second Screen:

10	20	30	40	50	60	70	80
		Wibaux University				MM/DD/YY	
		Instructor File Maintenance					
		Your name here					
		Social Security number . . 666666666					
		First name BBBBBBBBBB					
		Last name BBBBBBBBBBBBBBBB					
		Department BBB					
		Salary 999,999.99					
		Rank B					
		Sex B					
		Date of hire 99/99/9999					
		Marital status B					
		Number of dependents 99					
		Tenured (Y/N) B					
		Preferred title B					
		When finished, press Enter to xxxxxx					
20		F3=Exit, F12=Cancel					

Where xxxxx represents the word
Add, Update, or Delete

Notes:

1. Be sure to copy WUINSTP into MYINSTP before starting, and to use MYINSTP in your code.
2. Since this is an interactive application is not strictly necessary to “refresh” the file in between program calls, since MYINSTP serves a test file. Just keep track of what changes, deletions, and additions you have made to the file so that you know if your program is working as you develop the application. And if at any point you want to start “fresh” then “refresh” the file content as you have done in past projects. Remember that the Data Transfer feature in ACS can be used to download the data into a spreadsheet for reference.
3. Watch the Project 6 Video.
4. Note that the author has provided much of the logic for this type of application, on pages 395-398 in Chapter 12. Be sure to read the “pseudocode” on page 395 as well. It is important that you study the book example closely to understand the processes that are taking place.
5. Because this application is updating a file, you will need to use techniques from Chapter 9 for record addition to the file, updating a record, and deleting a record. These techniques are also included in the author’s examples for this chapter.
 - a. Users should NOT be allowed to change an existing Social Security Number
 - b. Users should NOT be allowed to add a record that would result in a duplicate Social Security Number.
 - c. Users should only be allowed to update or delete records that actually exist.
6. Data validation is an important aspect of this project. See the WUINSTP Record Layout above for the data validation rules that should be present in your screen DDS to enforce data integrity. Since these rules were NOT put into the original physical files (PF) definition (as you did for your project in CIS121 for example) the screen definition will need to include them, as we see in the author’s examples as well.
7. Extra credit for using RPG logic instead of the VALUES clause to enforce data integrity, as mentioned in the video above.
8. Notes on handling the date: the date on the screen can be set up as a “native” date in *USA format so that our users understand it. However, the date in the database file is a “legacy” date, one that is simply an 8-digit number, in ccyyymmdd format, such as 20140217 for February 7, 2014. Therefore it is necessary to convert the date to/from various formats in this application. Dates were covered in the first course. Here are some of the basics:

On DDS Screen Definition, the date can be shown as a native date, in *USA format:

DATEIN L B 10 33DATEFMT (*USA)

In the Instructor master file, the date is "legacy"-- 8S 0 (see Appendix F) therefore we must convert it from native to legacy/decimal (material from "Dates" chapter in first course) **before UPDATING or WRITING to the file in the RPG program:**

```
HIRDAT = %DEC (DATEIN) ; //HIRDAT is name of database field – see %DEC in textbook
                        // DATEIN is the native date from the screen in *USA format
```

When retrieving the date from the file, it will be in legacy/decimal form, so we must convert it to native form for the screen:

```
DATEIN = %DATE (HIRDAT) ; // DATEIN is the native date in *USA format for the screen
                        // HIRDAT is the legacy date we are converting before displaying in
                        // either change or delete mode – see %DATE in textbook
```

To Initialize a native date, *Blanks and *Zeroes will not work. Instead, use *Lval:

```
DATEIN = *Lval ; // sets our native date to 01/01/0001
```

Testing your program thoroughly is essential.

- Test F3 on both screens; F3 is an exit without making any changes, additions or deletions to the file
- Test F12 on second screen, it should return to first screen without performing any disk file changes, additions, or deletions. F12 means your user changed their mind!
- First screen: only accept A, C, or D
- Second screen:
 - Validation, only accept valid entries for
 - Rank
 - Sex
 - Marsts
 - Tenure
 - Title
 - Social Security Number should be output only, do not allow user to change it
- See date notes above
- Addition Testing
 - Good addition
 - Bad addition – try to add a record that already exists, program should not allow this
 - Check your data file contents
- Update Testing
 - Good update – only if record exists
 - Bad update – program should not allow update of a record that does not exist
 - Check your data file contents
- Delete Testing
 - Good delete – only if record exists
 - Bad delete – program should not allow delete of a record that does not exist
 - Check your data file contents